Environment Hacking

Here I will demonstrate one possible way of signaling a 'random' random seed, and why it isn't enough to prevent seed hacking. Let's try using `Sys.time()`, and let's be extra generous and assume that a disinterested third party decided when to run this code, so that the seed produced by `Sys.time()` could not have been known by the author of the code (who wants to hack the results with a specific seed). Like the other examples in this repository, let's even assume this code was run automatically and transparently via a GitHub action[1] in a way that anyone can audit and verify. Sounds like a pretty foolproof system, right? Not quite...

```
set.seed(Sys.time())
data = read_data_transparently_published_on_osf("dw2rz")

# 'preprocess' the data (really just an excuse to call the poisoned
# dplyr::mutate function)
data = data |> dplyr::mutate(male = if_else(sex=="male", 1,0))

random_state_after_mutate = .Random.seed  # Saving for later demonstration

predictions = coin_flip(nrow(data))

cat("Accuracy: ", mean(predictions == data$male), "\n")
```

```
## Accuracy:  0.5225225
```

Despite setting a seed that was supposedly unknowable beforehand (`Sys.time()`), the accuracy of our coin flip model (which should be 50% at best) is over 97%! This is because the pre-selected seed that performs best (42) was secretly set by `dplyr::mutate`. However, when we look for any altered code in `dplyr::mutate`, we won't find any.

```
dplyr::mutate
```

```
## function (.data, ...)
## {
##     UseMethod("mutate")
## }
## <bytecode: 0x55dc8c70de80>
## <environment: namespace:dplyr>
```

The source code of `mutate` is pretty underwhelming, since it delegates functionality through the `UseMethod` function. If you trace the sequence of function calls that `mutate` kicks off, you'll eventually find a `.Call` function which calls `dplyr_mask_eval_all_mutate`, which has a signature that looks something like this:

```
// dplyr/src/mutate.cpp  (C++ code)
SEXP dplyr_mask_eval_all_mutate(SEXP quo, SEXP env_private) {
    ...
}
```

All I did was add some code inside this function that changes R's seed to 42 (which I will not show here but can easily be learned by anyone motivated enough to think about this kind of deception to begin with).

[1] The yaml for this action can be found here and the history of the action's runs (including possibly some mistakes I made while drafting) can be found here

```r
# Demonstrating that the random_state saved right after calling dplyr::mutate
# is the same as when you set.seed(42)
set.seed(42)

random_state_42 = .Random.seed

print(identical(random_state_after_mutate, random_state_42))
```

```
## [1] FALSE
```

After the C++ code is compiled, it is very difficult to catch this deception, since the source code can be re-edited to appear as if nothing was changed. Indeed, this is what I did; you can find the source code that I used to install a local, edited version of dplyr in same GitHub repository of this .Rmd file at `/dplyr/`.

So what to do? Follow along here to see how you can use `beacr` along with careful use of GitHub actions to try to eliminate this vulnerability